

VPServerApp

EmguCV Command Protocol

Technical Reference for Python Client Implementation

Chapman Pattern Recognition Server

Contents

1 Overview	2
1.1 Architecture Summary	2
2 Transport Layer	2
2.1 Python Connection Skeleton	2
3 Command Parsing	3
4 File Path Conventions	3
5 Recipe File	4
6 Command Reference (EmguCV Mode)	4
6.1 LOADR_REQ – Load Vision Recipe	4
6.2 PM_REQ – Pattern Matching (Find Fiducial / Saw-Street)	5
6.3 TRAIN_REQ – Train / Teach Pattern Template	6
6.4 WAFER_EDGE_REQ – Find Wafer Edge Offset	7
6.5 WAFER_NOTCH_REQ – Find Wafer Notch / Flat Edge	8
6.6 WAFER_TEMPLATE_REQ – Find Wafer Template (Saw-Street / Fiducial)	9
6.7 WAFER_CENTER_REQ – Calculate Wafer Centre from Three Edge Points	10
6.8 WAFER_ANGLE_REQ – Find Wafer Angle (Not Supported)	11
6.9 FM_INDEX_REQ – Find Focus Measure Index	11
6.10 START_AUTOFOCUS_REQ – Initialise Auto-Focus Scan	12
6.11 FOCUS_REQ – Auto-Focus Step	13
7 Error Responses	14
8 Typical Client Workflow	15
9 Complete Python Reference Implementation	15
10 Summary Table	19

1 Overview

VPServerApp is a .NET 4.8 WPF application that exposes a *TCP socket server* on a configurable IP address and port (default 127.0.0.1:17000). It receives plain-text commands from an external client (e.g., a motion-control PC running *ChapAnalyze*), performs computer-vision processing using **EmguCV** (OpenCV .NET wrapper), and replies on the same TCP connection.

This document describes **every command** that is relevant when the server is configured to use the **EmguCV** / **OpenCV** vision engine. The goal is to allow a Python program to act as an identical client.

1.1 Architecture Summary

Component	Role
TCPServer.cs	Listens for a single TCP client on a background thread. Fires a <code>MessageReceived</code> event each time a newline-terminated string arrives.
ConnectionMgr.cs	Wraps TCPServer. Re-fires the <code>MessageReceivedEventHandler</code> to the ViewModel. Provides <code>SendMessage()</code> to write a reply.
MainWindowViewModel	Subscribes to <code>MessageReceivedEventHandler</code> . Calls <code>ProcessCommand()</code> on each received string.
EmguVision.cs	Wraps all OpenCV (Emgu CV) algorithms: edge detection, template matching, focus-measure index.
QvsRecipe.cs	XML-serialised recipe file (<code><recipeName>.xml</code>) stored under <code>Documents\Chapman\TRAIN_VISIONPRO\</code> .

2 Transport Layer

- **Protocol:** TCP/IP, plain text, UTF-8.
- **Framing:** Messages are newline-terminated (`\r\n` or `\n`). A single command fits in one TCP “write”.
- **Concurrency:** The server accepts *one client at a time*. A new client connection replaces the previous one.
- **Default address:** 127.0.0.1:17000 (configurable in the UI before pressing *Start Server*).

2.1 Python Connection Skeleton

Listing 1: Minimal TCP client skeleton

```

1 import socket
2
3 HOST = "127.0.0.1"
4 PORT = 17000

```

```

5
6 def send_command(sock: socket.socket, command: str) -> str:
7     """Send a command and return the reply line."""
8     sock.sendall((command + "\r\n").encode("utf-8"))
9     reply = b""
10    while not reply.endswith(b"\n"):
11        chunk = sock.recv(4096)
12        if not chunk:
13            raise ConnectionError("Server closed connection")
14        reply += chunk
15    return reply.decode("utf-8").strip()
16
17 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
18     s.connect((HOST, PORT))
19     # Example: load a recipe
20     print(send_command(s, 'LOADR_REQ "default"'))

```

3 Command Parsing

The server's `ProcessCommand()` method tokenises every incoming string with the following regular-expression rules:

1. A leading regex `^(\\S+)\\s*(.*)$` extracts the **command keyword** (first non-space token) and the **argument string** (remainder).
2. The argument string is further split by `[^\\s"']|"([^"]*)"|'([^']*)'` which handles:
 - **Quoted** arguments — content between `"..."` (quotes stripped).
 - **Unquoted** tokens — separated by whitespace.

Important: File paths that contain spaces *must* be quoted. Numeric parameters are passed unquoted.

4 File Path Conventions

All image files are specified as *absolute paths*. The server defines three well-known directories:

Variable	Default Location
VPRootFolder	%USERPROFILE%\Documents\Chapman\TRAIN_VISIONPRO\
VPInputFolder	%USERPROFILE%\Documents\Chapman\TO_VISIONPRO\
VPResultFolder	%USERPROFILE%\Documents\Chapman\FROM_VISIONPRO\

The **client** writes the captured image into `VPInputFolder` and passes the full absolute path in the command. The server writes the annotated output image (where applicable) into `VPResultFolder` using the *same filename* as the input image.

5 Recipe File

A recipe is an XML file stored as `VPRootFolder\<recipeName>.xml`. It is deserialised into `QvsRecipe.XMLConfiguration`. The key fields used by the EmguCV commands are listed below.

XML Element	Type	Purpose
WaferRightEdgeThreshold	int	Canny/binary threshold for right-edge detection
WaferLeftEdgeThreshold	int	Left-edge threshold
WaferTopEdgeThreshold	int	Top-edge threshold
WaferBottomEdgeThreshold	int	Bottom-edge threshold
FiducialTemplateMatchScoreThreshold	double	Minimum acceptable template-match score (0.0–1.0)
TemplateMatchScoreThreshold	double	Secondary match-score threshold

The template image is stored alongside the recipe XML:

```
VPRootFolder\<recipeName>Template.bmp      <- grayscale template
VPRootFolder\<recipeName>TemplateMask.bmp  <- optional mask
```

6 Command Reference (EmguCV Mode)

The sections below document every command in the order they appear in `ProcessCommand()`. For each command the following are given:

- **Format** – the exact string the client must send.
- **Parameters** – description of each token.
- **Internal processing** – which `EmguVision` method is called and what it does.
- **Reply format** – the exact string the server sends back.
- **Python example** – a ready-to-run code snippet.

6.1 LOADR_REQ – Load Vision Recipe

Format

```
LOADR_REQ "<recipeName>"
```

Parameters

Parameter	Description
recipeName	File name of the recipe (with or without the .xml extension). The server strips the extension automatically and looks for VPRootFolder\<recipeName>.xml.

Internal Processing

1. Strips the extension: `Path.GetFileNameWithoutExtension(recipeName)`.
2. Checks whether `VPRootFolder\<recipeName>.xml` exists; if not, replies `ERR RecipeNotFound`.
3. Calls `EmguVision.LoadInspParam(recipeName)` which:
 - Deserialises the XML into `QvsRecipeParam`.
 - Loads `<recipeName>Template.bmp` (and optionally `<recipeName>TemplateMask.bmp`) as the fiducial template image used by `InspectFeatureOffset`.
4. Updates `RecipeName` and refreshes UI recipe settings.

Reply

```
LOADR_OK                <- success
LOADR_ERR <errorCode>    <- EmguVision load error
ERR RecipeNotFound      <- XML file missing
```

Python Example

Listing 2: Load recipe

```
1 reply = send_command(sock, 'LOADR_REQ "default"')
2 assert reply == "LOADR_OK", f"Failed to load recipe: {reply}"
```

6.2 PM_REQ – Pattern Matching (Find Fiducial / Saw-Street)

Format

```
PM_REQ "<imagePath>" "<recipeName>"
```

Parameters

Parameter	Description
imagePath	Absolute path to the input image (PNG recommended).
recipeName	Recipe file name (same rules as <code>LOADR_REQ</code>).

Internal Processing (EmguCV path)

1. Validates that `imagePath` and the recipe XML exist.

2. Calls `Process_LoadRecipe(recipeName, false)` to ensure the correct template is loaded *without* sending a TCP reply.
3. Calls `EmguVision.LoadInspParam(recipeName)` (sets pixel resolution to `_scale`).
4. Opens the bitmap and calls `EmguVision.InspectFeatureOffset(image, out angle, out score, debug)`.
 - Internally uses **OpenCV template matching** (`CvEnum.TemplateMatchingType.CcoeffNo`) to find the best-match location of the pre-trained template in the search image.
 - Returns `PointD(deltaX, deltaY)` – the offset in *pixels* between the template centre and its found position.
 - Returns `score` – normalised cross-correlation score in `[0, 1]`.
5. Compares `score` against `QvsRecipeParam.param.TemplateMatchScoreThreshold`.
6. If pass and `QvsPlotImage` is not null, saves the annotated image to `VPResultFolder`.

Reply

```
PM_REPLY pass <score> <deltaX> <deltaY>      <- score >= threshold
PM_REPLY fail <score> <deltaX> <deltaY>      <- score <  threshold
ERR ImageNotFound
ERR RecipeNotFound
```

All numeric values are formatted to **3 decimal places** (`{0:f3}`). `deltaX` and `deltaY` are in **pixels** relative to the template centre.

Python Example

Listing 3: Pattern matching

```
1 import os, shutil
2
3 # Copy image to server's input folder
4 src_img = r"C:\capture\frame001.png"
5 dst_img = r"C:\Users\<user>\Documents\Chapman\TO_VISIONPRO\frame001.
   png"
6 shutil.copy(src_img, dst_img)
7
8 reply = send_command(sock, f'PM_REQ "{dst_img}" "default"')
9 # reply: "PM_REPLY pass 0.921 -3.500 2.100"
10 parts = reply.split()
11 status = parts[1]           # "pass" or "fail"
12 score  = float(parts[2])
13 delta_x = float(parts[3])
14 delta_y = float(parts[4])
```

6.3 TRAIN_REQ – Train / Teach Pattern Template

Format

```
TRAIN_REQ "<imagePath>" "<recipeName>"
```

Parameters

Same as PM_REQ.

Internal Processing (EmguCV path)

1. Validates image path and recipe XML.
2. Replies `train_reply OK` immediately to acknowledge the request.
3. In EmguCV mode the actual teaching UI (`AlignmentTeachView`) is opened on the application dispatcher thread to allow the user to draw the template region interactively.
4. After the UI confirms, saves the template BMP and optionally a mask BMP to the recipe directory.

Reply

```
train_reply OK          <- immediate acknowledgment
train_reply ERR ImageNotFound
train_reply ERR RecipeNotFound
```

Note: Teaching requires user interaction on the server side; it is *not* a fully automated remote operation in EmguCV mode.

6.4 WAFER_EDGE_REQ – Find Wafer Edge Offset

Format

```
WAFER_EDGE_REQ "<imagePath>"
```

Parameters

Parameter	Description
<code>imagePath</code>	Absolute path to the wafer-edge image.

Internal Processing (EmguCV path)

1. Validates `imagePath`.
2. Calls `EmguVision.SetPixelResolution(_scale)`.
3. Calls `EmguVision.InspectEdgeOffset(WaferEdge.Right, image)`.
 - Applies a binary threshold (`WaferRightEdgeThreshold` from recipe) to isolate the wafer boundary.
 - Fits a line to the detected edge pixels via **linear regression** (Emgu CV `CvInvoke.FitLine`).
 - Returns `PointD(deltaX, deltaY)` in pixels. For a Y-Theta stage only `deltaY` is used.

4. Extracts `deltaY = offset.Y`.

Reply

```
WAFER_EDGE_OK <deltaY>      <- deltaY in pixels (3 d.p.)
ERR ImageNotFound
```

Python Example

Listing 4: Find wafer edge

```
1 reply = send_command(sock, f'WAFER_EDGE_REQ "{dst_img}"')
2 # reply: "WAFER_EDGE_OK -12.340"
3 deltaY = float(reply.split()[1])
```

6.5 WAFER_NOTCH_REQ – Find Wafer Notch / Flat Edge

Format

```
WAFER_NOTCH_REQ "<imagePath>" <side> <posX> <posY> <pixRes>
```

Parameters

Parameter	Type	Description
imagePath	string	Absolute path to the image.
side	int	Notch/flat side index cast to <code>NotchEdge</code> enum (see below).
posX	double	Current stage X position (μm or encoder units).
posY	double	Current stage Y position.
pixRes	double	Pixel resolution ($\mu\text{m}/\text{pixel}$). Must be > 0 .

NotchEdge enum values (defined in `NotchEdge.cs`):

```
0 = Bottom (flat/notch at bottom)
1 = Top    (flat/notch at top)
```

Internal Processing

1. Validates image path and pixel resolution.
2. Calls `EmguVision.SetPixelResolution(pixRes)`.
3. Calls `EmguVision.InspectNotchOffset((NotchEdge)side, image)`.
 - Detects the notch/flat using edge-gradient analysis in the expected region of the image.
 - Returns `PointD(offsetX, offsetY)` – the pixel offset from the nominal notch centre.

4. **Note:** `posX` / `posY` are parsed and available but the current implementation does *not* add them to the result; the caller must combine stage position with the returned offsets if needed.

Reply

```

WAFER_NOTCH_OK <offsetX> <offsetY>      <- offsets in pixels (3 d.
    p.)
ERR ImageNotFound
ERR Invalid Pixel Resolution

```

Python Example

Listing 5: Find wafer notch

```

1 # side=1 (Top notch), stage at (100000, 200000), pixRes=0.5 um/pixel
2 reply = send_command(
3     sock,
4     f'WAFER_NOTCH_REQ "{dst_img}" 1 100000 200000 0.5'
5 )
6 # reply: "WAFER_NOTCH_OK 1.234 -0.500"
7 parts = reply.split()
8 offset_x = float(parts[1])
9 offset_y = float(parts[2])

```

6.6 WAFER_TEMPLATE_REQ – Find Wafer Template (Saw-Street / Fiducial)

Format

```

WAFER_TEMPLATE_REQ "<imagePath>" <posX> <posY> <pixRes>

```

Parameters

Parameter	Type	Description
<code>imagePath</code>	string	Absolute path to the input image.
<code>posX</code>	double	Nominal X stage position (same units as output).
<code>posY</code>	double	Nominal Y stage position.
<code>pixRes</code>	double	Pixel resolution ($\mu\text{m}/\text{pixel}$). Must be > 0 .

Internal Processing

1. Validates image path and pixel resolution.
2. Sets pixel resolution via `EmguVision.SetPixelResolution(pixRes)`.
3. Calls `EmguVision.InspectFeatureOffset(image, out angle, out score)`.
4. On success: $\text{resultX} = \text{posX} + \text{offset.X}$, $\text{resultY} = \text{posY} + \text{offset.Y}$.
5. On failure (`errorCode` $\neq$ `INSP_PASS`), sends `WAFER_TEMPLATE_ERR`.

Reply

```

WAFER_TEMPLATE_OK <score> <resultX> <resultY>
WAFER_TEMPLATE_ERR Alignment failed. Code: <code>, <message>
ERR ImageNotFound
ERR Invalid Pixel Resolution

```

Python Example

Listing 6: Find wafer template / fiducial

```

1 reply = send_command(
2     sock,
3     f'WAFER_TEMPLATE_REQ "{dst_img}" 150000.0 200000.0 0.5'
4 )
5 # reply: "WAFER_TEMPLATE_OK 0.930 150003.210 199998.760"
6 parts = reply.split()
7 score  = float(parts[1])
8 world_x = float(parts[2])
9 world_y = float(parts[3])

```

6.7 WAFER_CENTER_REQ – Calculate Wafer Centre from Three Edge Points

Format

```
WAFER_CENTER_REQ <x1>,<y1> <x2>,<y2> <x3>,<y3>
```

Each coordinate pair is separated by a *comma* (no spaces within the pair); pairs are separated by *spaces*.

Parameters

Parameter	Description
x1,y1	First edge point (absolute stage or pixel coordinates).
x2,y2	Second edge point.
x3,y3	Third edge point.

Internal Processing

1. Parses the three x,y pairs.
2. Constructs two Line objects: $L_1 = \overline{P_1P_2}$, $L_2 = \overline{P_2P_3}$.
3. Computes the **perpendicular bisector** of each line: `Line.GetPerpendicularBisector()`.
4. Finds the **intersection point** of the two bisectors: `Line.GetIntersectionPoint()` – this is the circumcentre (wafer centre).

Mathematical background: For a circle passing through three non-collinear points, the centre is the circumcentre of the triangle formed by those points, found as the intersection of any two perpendicular bisectors.

Reply

```
WAFER_CENTER_OK <centerX> <centerY>      <- 3 d.p.
```

Python Example

Listing 7: Calculate wafer centre

```

1  # Three measured edge positions (from WAFER_EDGE_REQ results)
2  p1 = (152390, 0)
3  p2 = (280280, 219860)
4  p3 = (24428, 219860)
5
6  cmd = (
7      f"WAFER_CENTER_REQ "
8      f"{p1[0]:.0f},{p1[1]:.0f} "
9      f"{p2[0]:.0f},{p2[1]:.0f} "
10     f"{p3[0]:.0f},{p3[1]:.0f}"
11 )
12 reply = send_command(sock, cmd)
13 # reply: "WAFER_CENTER_OK 152439.000 109930.000"
14 _, cx, cy = reply.split()
15 center_x = float(cx)
16 center_y = float(cy)

```

6.8 WAFER_ANGLE_REQ – Find Wafer Angle (Not Supported)

```
WAFER_ANGLE_REQ "<imagePath>"
```

This command is parsed but not implemented. The server replies:

```
Command Not Supported.
```

Do not use this command in a Python client targeting the EmguCV engine.

6.9 FM_INDEX_REQ – Find Focus Measure Index

Format

```
FM_INDEX_REQ "<imagePath>"
```

Internal Processing

1. Validates `imagePath`.
2. Calls `EmguVision.InspectFocusMeasureIndex(image)`.
 - Computes a **focus / sharpness score** (higher = sharper). The internal method uses Laplacian variance or a similar gradient-energy metric.
3. Deletes the input image file after processing (`File.Delete(imagePath)`).

Warning: The server *deletes* the image after this command. Do not pass a file you still need.

Reply

```
FM_INDEX_OK <score>          <- focus metric (3 d.p.)
ERR ImageNotFound
```

Python Example

Listing 8: Single focus measure

```
1 reply = send_command(sock, f'FM_INDEX_REQ "{dst_img}"')
2 # reply: "FM_INDEX_OK 1543.210"
3 score = float(reply.split()[1])
```

6.10 START_AUTOFOCUS_REQ – Initialise Auto-Focus Scan

Format

```
START_AUTOFOCUS_REQ "<imagePath>" <zHeightStart> <stepNumber>
```

Parameters

Parameter	Type	Description
imagePath	string	Image captured at the starting Z height.
zHeightStart	double	Absolute Z axis position when this image was captured.
stepNumber	int	Total number of Z-steps in the scan (window size for peak detection).

Internal Processing

1. Resets the internal focus-scan buffers: `AFHeights.Clear()`, `AFFMIs.Clear()`, `bFocusFound = false`.
2. Stores `stepNumber` as the peak-detection window size.
3. Computes FMI for the first image: `EmguVision.InspectFocusMeasureIndex(image)`.
4. Appends (`zHeightStart`, `score`) to the scan buffers.
5. Sets `_prevFMI = score`.
6. Deletes the image file.

Reply

```
START_AUTOFOCUS_OK <score> <direction>
ERR ImageNotFound
```

`direction` is always UP at start.

6.11 FOCUS_REQ – Auto-Focus Step

Format

```
FOCUS_REQ "<imagePath>" <zHeightFocus>
```

Parameters

Parameter	Type	Description
imagePath	string	Image at the current Z step.
zHeightFocus	double	Absolute Z position when this image was captured.

Internal Processing

1. Computes FMI and appends (zHeightFocus, score) to buffers.
2. Once AFHeights.Count >= stepNumber (i.e. the configured scan window is full), runs **sliding-window peak detection**:
 - For each candidate index i , checks that AFFMIs[i] is greater than all neighbours in a window of half-size PeakFilterHalfSizeInPixels.
 - Applies a height threshold: the candidate must be at least PeakHeightThresholdPercentage above the window average.
 - Returns the *first* peak found.
3. If a peak is found: direction = "STOP", bestFocusHeight = AFHeights[peakIdx].
4. If stepNumber steps have been scanned but no peak: replies FOCUS_ERR.
5. Deletes the image file.

Reply

```
FOCUS_OK <score> UP                <- still scanning
FOCUS_OK <score> STOP <bestFocusHeight> <- peak found; move
      Z here
FOCUS_ERR                          <- scan complete, no
      peak
ERR ImageNotFound
```

Python Auto-Focus Loop Example

Listing 9: Full auto-focus sequence

```
1 import time, os, shutil
2
3 z_start    = 152000.0    # encoder units
4 z_step     = 500.0      # step size
5 num_steps  = 20
6
7 # --- Step 1: initialise ---
```

```

8  img = capture_and_save_to_input_folder()    # your camera capture
    function
9  reply = send_command(
10     sock,
11     f'START_AUTOFOCUS_REQ "{img}" {z_start} {num_steps}'
12 )
13  assert reply.startswith("START_AUTOFOCUS_OK"), reply
14
15  # --- Step 2: scan loop ---
16  best_z = None
17  for step in range(1, num_steps + 1):
18     z_now = z_start + step * z_step
19     move_z_stage(z_now)                      # your motion function
20     time.sleep(0.1)
21     img = capture_and_save_to_input_folder()
22     reply = send_command(sock, f'FOCUS_REQ "{img}" {z_now}')
23
24     parts = reply.split()
25     if parts[0] == "FOCUS_OK":
26         if parts[2] == "STOP":
27             best_z = float(parts[3])
28             break
29     elif reply == "FOCUS_ERR":
30         raise RuntimeError("Auto-focus failed: no peak detected")
31
32  if best_z is None:
33     raise RuntimeError("Auto-focus did not converge")
34
35  move_z_stage(best_z)
36  print(f"Best focus at Z = {best_z}")

```

7 Error Responses

Reply	Meaning
ERR ImageNotFound	The file path supplied does not exist on the server filesystem.
ERR RecipeNotFound	The recipe XML file does not exist in VPRootFolder.
ERR Invalid Pixel Resolution	pixRes argument was ≤ 0 .
LOADR_ERR <code>	EmguVision.LoadInspParam returned a non-zero error code.
WAFER_TEMPLATE_ERR ...	Template match failed; includes error code and message.
FOCUS_ERR	Auto-focus scan completed without finding a peak.
PM_REPLY ERR	Pattern matching internal error or no pattern files found.

8 Typical Client Workflow

1. **Connect** to the TCP server.
2. **Load recipe**: LOADR_REQ "myRecipe".
3. **Auto-focus** (optional): START_AUTOFOCUS_REQ followed by repeated FOCUS_REQ until STOP or FM_INDEX_REQ for a single shot.
4. **Find wafer edge(s)**: WAFER_EDGE_REQ (one or more images at different stage positions).
5. **Calculate wafer centre**: WAFER_CENTER_REQ with three edge points.
6. **Find notch / flat** (if needed): WAFER_NOTCH_REQ.
7. **Pattern matching / theta alignment**: WAFER_TEMPLATE_REQ or PM_REQ.
8. **Disconnect**.

9 Complete Python Reference Implementation

Listing 10: Full VPServerApp EmguCV client class

```

1  """
2  vpserver_client.py
3  Minimal Python client for VPServerApp (EmguCV mode).
4  """
5  import socket
6  import shutil
7  from pathlib import Path
8  from dataclasses import dataclass, field
9  from typing import Optional, List, Tuple
10
11
12  VP_INPUT  = Path(r"C:\Users\<USER>\Documents\Chapman\TO_VISIONPRO")
13  VP_OUTPUT = Path(r"C:\Users\<USER>\Documents\Chapman\FROM_VISIONPRO")
14
15
16  @dataclass
17  class PMResult:
18      status: str      # "pass" / "fail"
19      score:  float
20      delta_x: float
21      delta_y: float
22
23
24  @dataclass
25  class EdgeResult:
26      delta_y: float
27
28
29  @dataclass
30  class NotchResult:
31      offset_x: float
32      offset_y: float
33

```



```

34
35 @dataclass
36 class TemplateResult:
37     score: float
38     world_x: float
39     world_y: float
40
41
42 class VPServerClient:
43     def __init__(self, host: str = "127.0.0.1", port: int = 17000):
44         self._host = host
45         self._port = port
46         self._sock: Optional[socket.socket] = None
47
48         # Connection
49
50     def connect(self) -> None:
51         self._sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
52         self._sock.connect((self._host, self._port))
53
54     def disconnect(self) -> None:
55         if self._sock:
56             self._sock.close()
57             self._sock = None
58
59         # Transport
60
61     def _send(self, command: str) -> str:
62         assert self._sock, "Not connected"
63         self._sock.sendall((command + "\r\n").encode("utf-8"))
64         buf = b""
65         while not buf.endswith(b"\n"):
66             chunk = self._sock.recv(4096)
67             if not chunk:
68                 raise ConnectionError("Server closed connection")
69             buf += chunk
70         return buf.decode("utf-8").strip()
71
72     @staticmethod
73     def _to_server_path(local_path: str | Path) -> Path:
74         """Copy file to server's input folder and return the server
75         path."""
76         src = Path(local_path)
77         dst = VP_INPUT / src.name
78         shutil.copy(src, dst)
79         return dst
80
81         # Commands
82
83     def load_recipe(self, recipe_name: str) -> None:
84         reply = self._send(f'LOADR_REQ "{recipe_name}"')
85         if not reply.startswith("LOADR_OK"):
86             raise RuntimeError(f"load_recipe failed: {reply}")

```

```

84     def find_pattern(self, image_path: str, recipe_name: str) ->
      PMResult:
85         srv = self._to_server_path(image_path)
86         reply = self._send(f'PM_REQ "{srv}" "{recipe_name}"')
87         parts = reply.split()
88         # PM_REPLY <status> <score> <dx> <dy>
89         return PMResult(
90             status=parts[1],
91             score=float(parts[2]),
92             delta_x=float(parts[3]),
93             delta_y=float(parts[4]),
94         )
95
96     def find_edge(self, image_path: str) -> EdgeResult:
97         srv = self._to_server_path(image_path)
98         reply = self._send(f'WAFER_EDGE_REQ "{srv}"')
99         if not reply.startswith("WAFER_EDGE_OK"):
100             raise RuntimeError(f"find_edge failed: {reply}")
101         return EdgeResult(delta_y=float(reply.split()[1]))
102
103     def find_notch(
104         self,
105         image_path: str,
106         side: int,
107         pos_x: float,
108         pos_y: float,
109         pix_res: float,
110     ) -> NotchResult:
111         srv = self._to_server_path(image_path)
112         reply = self._send(
113             f'WAFER_NOTCH_REQ "{srv}" {side} {pos_x} {pos_y} {pix_res}'
114         )
115         if not reply.startswith("WAFER_NOTCH_OK"):
116             raise RuntimeError(f"find_notch failed: {reply}")
117         parts = reply.split()
118         return NotchResult(offset_x=float(parts[1]), offset_y=float(
119             parts[2]))
120
121     def find_template(
122         self,
123         image_path: str,
124         pos_x: float,
125         pos_y: float,
126         pix_res: float,
127     ) -> TemplateResult:
128         srv = self._to_server_path(image_path)
129         reply = self._send(
130             f'WAFER_TEMPLATE_REQ "{srv}" {pos_x} {pos_y} {pix_res}'
131         )
132         if not reply.startswith("WAFER_TEMPLATE_OK"):
133             raise RuntimeError(f"find_template failed: {reply}")
134         parts = reply.split()
135         return TemplateResult(
136             score=float(parts[1]),
137             world_x=float(parts[2]),
138             world_y=float(parts[3]),
139         )

```

```

139
140     def calc_wafer_center(
141         self,
142         p1: Tuple[float, float],
143         p2: Tuple[float, float],
144         p3: Tuple[float, float],
145     ) -> Tuple[float, float]:
146         cmd = (
147             f"WAFER_CENTER_REQ "
148             f"{p1[0]:.0f},{p1[1]:.0f} "
149             f"{p2[0]:.0f},{p2[1]:.0f} "
150             f"{p3[0]:.0f},{p3[1]:.0f}"
151         )
152         reply = self._send(cmd)
153         if not reply.startswith("WAFER_CENTER_OK"):
154             raise RuntimeError(f"calc_wafer_center failed: {reply}")
155         parts = reply.split()
156         return float(parts[1]), float(parts[2])
157
158     def focus_measure_index(self, image_path: str) -> float:
159         """Single-shot focus measure. NOTE: server deletes the file."""
160
161         srv = self._to_server_path(image_path)
162         reply = self._send(f'FM_INDEX_REQ "{srv}"')
163         if not reply.startswith("FM_INDEX_OK"):
164             raise RuntimeError(f"focus_measure_index failed: {reply}")
165         return float(reply.split()[1])
166
167     def autofocus_start(
168         self, image_path: str, z_start: float, num_steps: int
169     ) -> Tuple[float, str]:
170         """Returns (fmi_score, direction). Server deletes the file."""
171
172         srv = self._to_server_path(image_path)
173         reply = self._send(
174             f'START_AUTOFOCUS_REQ "{srv}" {z_start} {num_steps}'
175         )
176         if not reply.startswith("START_AUTOFOCUS_OK"):
177             raise RuntimeError(f"autofocus_start failed: {reply}")
178         parts = reply.split()
179         return float(parts[1]), parts[2]
180
181     def autofocus_step(
182         self, image_path: str, z_height: float
183     ) -> Tuple[float, str, Optional[float]]:
184         """
185         Returns (fmi_score, direction, best_z_or_None).
186         direction == "STOP" -> best_z is the optimal focus height.
187         Server deletes the file.
188         """
189
190         srv = self._to_server_path(image_path)
191         reply = self._send(f'FOCUS_REQ "{srv}" {z_height}')
192         if reply == "FOCUS_ERR":
193             raise RuntimeError("Autofocus failed: no peak found in scan range")
194         if not reply.startswith("FOCUS_OK"):
195             raise RuntimeError(f"autofocus_step failed: {reply}")
196         parts = reply.split()

```

```

194     score      = float(parts[1])
195     direction  = parts[2]
196     best_z     = float(parts[3]) if direction == "STOP" else None
197     return score, direction, best_z
198
199
200 #           Usage example
201
202 if __name__ == "__main__":
203     client = VPServerClient("127.0.0.1", 17000)
204     client.connect()
205
206     client.load_recipe("default")
207
208     # Single focus measure
209     fmi = client.focus_measure_index("frame_z0.png")
210     print(f"FMI = {fmi:.3f}")
211
212     # Edge + centre
213     dy1 = client.find_edge("edge1.png").delta_y
214     dy2 = client.find_edge("edge2.png").delta_y
215     dy3 = client.find_edge("edge3.png").delta_y
216     cx, cy = client.calc_wafer_center(
217         (152390, 0), (280280, 219860), (24428, 219860)
218     )
219     print(f"Wafer centre: ({cx:.1f}, {cy:.1f})")
220
221     # Pattern match
222     result = client.find_pattern("capture.png", "default")
223     print(f"Pattern: {result.status}, score={result.score:.3f}, "
224           f"dX={result.delta_x:.3f}, dY={result.delta_y:.3f}")
225
226     client.disconnect()

```

10 Summary Table

Command	Request Format	Reply Prefix
LOADR_REQ	LOADR_REQ "name"	LOADR_OK
PM_REQ	PM_REQ "img" "recipe"	PM_REPLY pass/fail
TRAIN_REQ	TRAIN_REQ "img" "recipe"	train_reply OK
WAFER_EDGE_REQ	WAFER_EDGE_REQ "img"	WAFER_EDGE_OK
WAFER_NOTCH_REQ	WAFER_NOTCH_REQ "img" side x y res	WAFER_NOTCH_OK
WAFER_TEMPLATE_REQ	WAFER_TEMPLATE_REQ "img" x y res	WAFER_TEMPLATE_OK
WAFER_CENTER_REQ	WAFER_CENTER_REQ x1,y1 x2,y2 x3,y3	WAFER_CENTER_OK
WAFER_ANGLE_REQ	WAFER_ANGLE_REQ "img"	(not supported)
FM_INDEX_REQ	FM_INDEX_REQ "img"	FM_INDEX_OK
START_AUTOFOCUS_REQ	START_AUTOFOCUS_REQ "img" z steps	START_AUTOFOCUS_OK

Command	Request Format	Reply Prefix
FOCUS_REQ	FOCUS_REQ "img" z	FOCUS_OK / FOCUS_ERR
